

Algoritmi-Modulo algoritmi per bioinformatica

Ordinamento in tempo lineare-Capitolo 8 CLRS

Roberto Posenato

versione 2026-2, 19/04/2026

- 1 Limiti inferiori per l'ordinamento
 - Idea generale
 - Un limite inferiore per il caso peggiore
- 2 CountingSort
 - Idea generale
 - Algoritmo
 - Complessità
- 3 BucketSort
 - Idea generale
 - Algoritmo
 - Complessità
- 4 Esercizi

*“Tell me and I forget, teach me and I remember,
involve me and I learn.”*

Benjamin Franklin

“A student who achieves knowledge through free investigation and spontaneous effort will be able to retain that knowledge and will have acquired a methodology that can serve for a lifetime.”

Jean Piaget

Scopo della lezione

- Fino a ora abbiamo considerato algoritmi di ordinamento basati sul confronto.
- In questa lezione si dimostrerà che tutti gli algoritmi di ordinamento basati sul confronto hanno complessità nel caso peggiore $\Omega(n \lg n)$.
- Per completare la rassegna degli algoritmi di ordinamento, si presenteranno poi due algoritmi che **non** sono basati sul confronto: counting sort e bucket sort .

Limiti inferiori per l'ordinamento

Idea generale

- Gli algoritmi di ordinamento basati sul confronto eseguono test del tipo $x < y$, $x == y$ e $x > y$.
- Gli ordinamenti per confronti possono essere rappresentati in termini di **alberi di decisione**.

Limiti inferiori per l'ordinamento

Idea generale

Definizione 1 (Alberi di decisione per algoritmi di ordinamento)

Assumendo per semplicità che l'input sia formato da elementi **distinti**, un albero di decisione è un albero binario **pieno**^a che rappresenta i confronti fra elementi di un input di dimensione n di un algoritmo di ordinamento. Ogni nodo considera una coppia di elementi (a, b) .

Il figlio di sinistra rappresenta il fatto che $(a \leq b)$.

Il figlio di destra rappresenta il fatto che $(a > b)$.

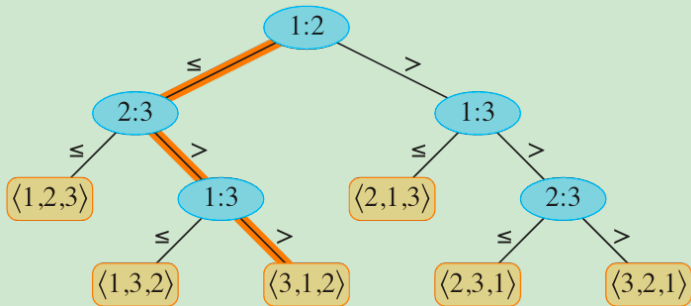
^aIn un albero binario **pieno** ogni nodo interno ha o 2 figli o 0 figli. Un albero binario **completo** è un albero pieno con tutte le foglie allo stesso livello.

Limiti inferiori per l'ordinamento

Idea generale

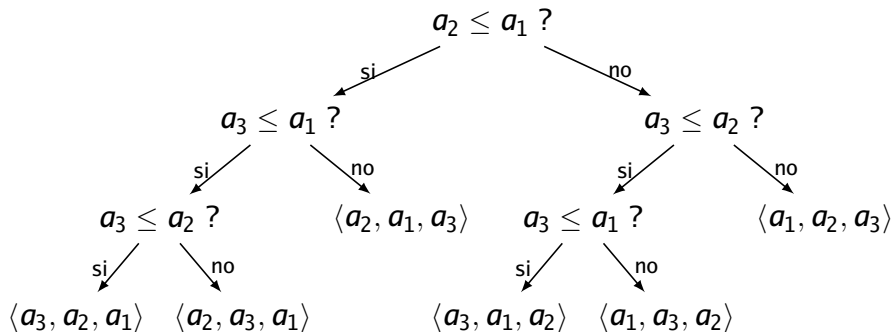
Esempio 1 (Albero di decisione per INSERTIONSORT su 3 elementi)

Con input $[6, 8, 5]$,
INSERTIONSORT esegue i
confronti secondo il percorso
evidenziato.



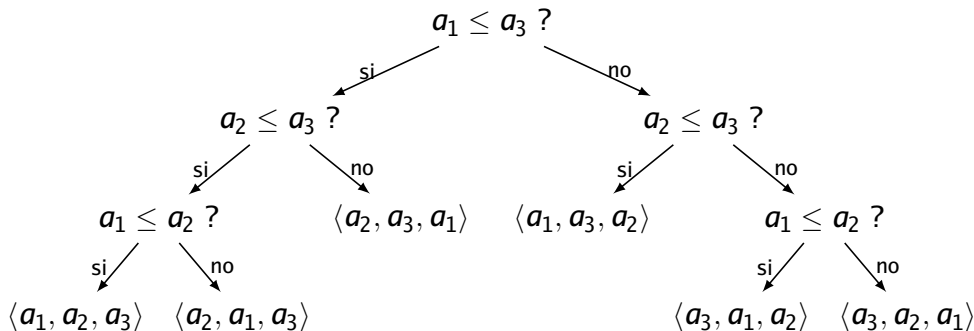
Albero di decisione di INSERTION-SORT su 3 elementi

Sia l'input $A = \langle a_1, a_2, a_3 \rangle$, con elementi distinti.



Albero di decisione di QUICKSORT deterministico su 3 elementi

Sia l'input $A = \langle a_1, a_2, a_3 \rangle$, con elementi distinti. La prima chiamata a PARTITION usa a_3 come pivot.



Limiti inferiori per l'ordinamento

Idea generale

- Ogni algoritmo di ordinamento per confronto deve essere in grado di produrre ogni permutazione di n elementi distinti.
- Con n elementi distinti, ci sono $n!$ permutazioni possibili.
- Ogni possibile permutazione deve essere rappresentata come foglia di un albero di decisione.
- Si considerano quindi solo alberi di decisione contenenti tutte le $n!$ permutazioni come foglie.

Limiti inferiori per l'ordinamento

Un limite inferiore per il caso peggiore

Si fissi un algoritmo di ordinamento per confronto e il suo albero di decisione per un dato n .

- La lunghezza del cammino **più lungo** dalla radice a una delle foglie rappresenta il numero di confronti nel caso peggiore.
- L'altezza dell'albero di decisione è uguale al numero di confronti nel caso peggiore.
- Un limite inferiore all'altezza degli alberi di decisione è quindi un limite inferiore sul tempo di esecuzione di qualsiasi algoritmo di ordinamento per confronti nel caso peggiore.
- Dato un albero di decisione con l foglie che corrisponde a un ordinamento di n elementi: deve essere $n! \leq l$.
- Un albero binario di altezza h non ha più di 2^h foglie: $\Rightarrow n! \leq l \leq 2^h$.
- $h \geq \lg(n!) = \Omega(n \lg n)$ (usando approssimazione di Stirling*).

* $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$

COUNTINGSORT

Idea generale

- Algoritmo di ordinamento **non basato sul confronto**.
- L'input **deve** essere un array d' **interi** nell'intervallo $[0 : k]$ per un k intero fissato.
- Idea: per ogni valore x , si conta quante volte compare e, con i conteggi cumulativi, si determina il blocco di posizioni finali riservato ai valori uguali a x .
- Per *contare*, è sufficiente, in un array $C[0 : k]$, aumentare di 1 l'elemento $C[x]$ quando si incontra x in input.
- La complessità è $\Theta(k + n)$, se $k = O(n)$, allora la complessità diventa $\Theta(n)$.

COUNTINGSORT

Algoritmo

Algoritmo: COUNTINGSORT(A, k)

Input: A array da ordinare, k massimo valore possibile per gli elementi di A

Output: B , array ordinato

```
1:  $B[1 : n]$  nuovo array
2:  $C[0 : k]$  nuovo array di supporto inizializzato a 0
3: for  $j = 1$  to  $A.length$  do
4:    $C[A[j]] ++$                                      //  $C[i] = \#$  volte  $i$  compare in  $A$ 
5: for  $i = 1$  to  $k$  do
6:    $C[i] = C[i] + C[i - 1]$                              //  $C[i] = \#$  volte valori  $\leq i$  compaiono in  $A$ 
   //  $C[i]$  indica quindi l'ultima posizione di  $B$  occupabile da un valore  $i$ 
7: for  $j = A.length$  to 1 do
8:    $B[C[A[j]]] = A[j]$ 
9:    $C[A[j]] --$  // Se ci sono più valori  $= A[j]$ , prossimo  $A[j]$  viene messo nella posizione precedente
10: return  $B$ 
```

COUNTINGSORT

Esempio

	1	2	3	4	5	6	7	8
A	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
C	2	0	2	3	0	1

(a)

	0	1	2	3	4	5
C	2	2	4	7	7	8

(b)

	1	2	3	4	5	6	7	8
B						3		

	0	1	2	3	4	5
C	2	2	4	6	7	8

(c)

	1	2	3	4	5	6	7	8
B		0					3	

	0	1	2	3	4	5
C	1	2	4	6	7	8

(d)

	1	2	3	4	5	6	7	8
B		0				3	3	

	0	1	2	3	4	5
C	1	2	4	5	7	8

(e)

	1	2	3	4	5	6	7	8
B	0	0	2	2	3	3	3	5

(f)

$A.length = 8, k = 5.$

(a) C dopo ciclo 2–3. (b) C dopo ciclo 4–5.

(c)–(e) alcuni passaggi del ciclo 6–7.

(f) B finale

COUNTINGSORT

Complessità

- Ciclo **for** righe 2–3 richiede tempo $\Theta(n)$.
- Ciclo **for** righe 4–5 richiede tempo $\Theta(k)$.
- Ciclo **for** righe 6–8 richiede tempo $\Theta(n)$.
- **Tempo totale** = $\Theta(k + n)$.
- Se $k = O(n)$, allora **Tempo totale** = $\Theta(n)$.

Algoritmo: COUNTINGSORT(A, k)

Input: A array, k max. val.

Output: B , array ordinato

```
1:  $C[0, \dots, k]$  nuovo array
2: for  $j = 1$  to  $A.length$  do
3:    $C[A[j]] ++$ 
4: for  $i = 1$  to  $k$  do
5:    $C[i] = C[i] + C[i - 1]$ 
6: for  $j = A.length$  to  $1$  do
7:    $B[C[A[j]]] = A[j]$ 
8:    $C[A[j]] --$ 
9: return  $B$ 
```

Nota: Il tempo è inferiore al limite $\Omega(n \lg n)$ perché questo algoritmo non si basa sul confronto!

- COUNTINGSORT è un algoritmo **stabile**: gli elementi con lo stesso valore mantengono la loro posizione relativa.
- Questo dipende dal fatto che l'ultimo ciclo scorre A da destra verso sinistra: a parità di valore, l'elemento che compare più a destra viene collocato per primo nella posizione finale più a destra.
- Per esempio: se un '2', chiamiamolo *primo2*, viene prima di un altro '2', anche nell'array finale *primo2* viene prima dell'altro 2.

BUCKETSORT

Idea generale

Assunzioni della versione presentata

- Ordina valori decimali.
- In CLRS si assume che i valori siano decimali positivi in $[0, 1)$, ma è una limitazione che si può eliminare con una semplice modifica (di costo lineare).
- Per ottenere un tempo di esecuzione medio $\Theta(n)$, si assume inoltre che i valori in input siano *estratti in modo indipendente e uniforme* da $[0, 1)$.

BUCKETSORT

Idea generale

Idea

- Si divide l'intervallo $[0, 1)$ in n intervalli della stessa dimensione, detti **bucket**
- Si distribuiscono gli n elementi nei bucket.
 - Grazie all'ipotesi di distribuzione uniforme, ciascun bucket non dovrebbe avere molti più elementi degli altri bucket.
- Si ordina ciascun bucket con INSERTIONSORT!
- Ogni bucket contiene valori di un intervallo più piccolo di quelli contenuti nei bucket successivi.
- Si riportano quindi i numeri dal primo bucket all'ultimo per ottenere l'ordinamento.

BUCKETSORT

Algoritmo

Algoritmo: BUCKETSORT(A)

Input: A array da ordinare

Output: C , array ordinato

1: $n = A.length$

2: $B[0 : n - 1]$ nuovo array di liste vuote

// Ogni lista $\rightarrow$ array di lungh. variabile

3: $C[1 : n]$ nuovo array vuoto

4: **for** $i = 1$ **to** n **do**

5: $B[\lfloor n \cdot A[i] \rfloor].ADD(A[i])$

// Inserisce $A[i]$ in testa al proprio bucket

6: **for** $i = 0$ **to** $n - 1$ **do**

7: $INSERTIONSORT(B[i])$

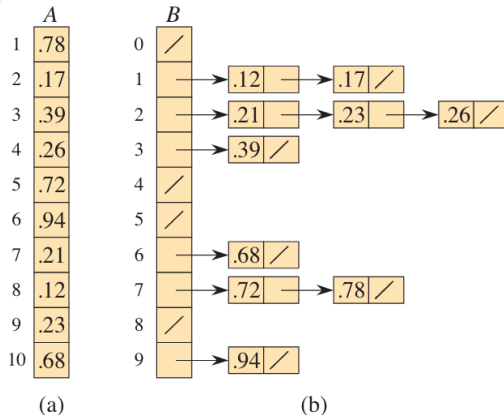
8: Aggiungi a C tutti i valori ordinati di $B[i]$

9: **return** C

* A e C hanno indici da 1 a n ; B da 0 a $n - 1$ compreso.

BUCKETSORT

Esempio



(a) array iniziale A

(b) array delle liste B dopo il secondo **for**

Con 10 bucket, la determinazione della lista (vettore) di B dove aggiungere l'elemento è facile: corrisponde alla prima cifra decimale.

- Si considerino due elementi: $A[i]$ e $A[j]$.
- Si supponga che $A[i] \leq A[j]$ (analogo ragionamento vale per $\geq$).
- Moltiplicando per n (positivo): $\lfloor nA[i] \rfloor \leq \lfloor nA[j] \rfloor$
- $A[i]$ viene messo nello stesso bucket o in quello precedente a $A[j]$.
- Se sono nello stesso bucket, INSERTIONSORT li ordinerà tra loro.
- Se sono in bucket diversi, istruzione 8 li aggiunge in modo ordinato nell'array finale.

BUCKETSORT

Complessità

- Ciclo **for** righe 4–5 richiede tempo $\Theta(n)$.
- Per ogni ciclo del **for** righe 6–8:
 - si esegue un INSERTIONSORT
 - si aggiungono i valori ordinati da INSERTIONSORT nell'array finale (riga 8).
- Il costo totale riga 8 su tutti gli n cicli è $\Theta(n)$ perché sono n in totale i valori da aggiungere all'array finale.
- Rimane da calcolare il tempo totale richiesto dalle n chiamate a INSERTIONSORT.

Algoritmo: BUCKETSORT(A)

Input: A array da ordinare

Output: C , array ordinato

```
1:  $n = A.length$ 
2:  $B[0 : n - 1]$  nuovo array di liste vuote
3:  $C[1 : n]$  nuovo array vuoto
4: for  $i = 1$  to  $n$  do
5:    $B[\lfloor n \cdot A[i] \rfloor].ADD(A[i])$ 
6: for  $i = 0$  to  $n - 1$  do
7:   INSERTIONSORT( $B[i]$ )
8:   Aggiungi a  $C$  tutti i valori ordinati di  $B[i]$ 
9: return  $C$ 
```

BUCKETSORT

Complessità: analisi caso medio

- n_i = variabile casuale “# elementi in $B[i]$ ”
- INSERTIONSORT sul bucket i ha complessità $O(n_i^2)$
- Complessità di BUCKETSORT è $T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$
- Caso ‘medio’: si calcola la dimensione media $E[n_i^2]$ di ciascun bucket i in quanto si assume che INSERTIONSORT richieda tempo $O(n_i^2)$ per bucket i .

$$\begin{aligned} E[T(n)] &= E \left[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2) \right] \\ &= \Theta(n) + \sum_{i=0}^{n-1} E \left[O(n_i^2) \right] \\ &= \Theta(n) + \sum_{i=0}^{n-1} O \left(E[n_i^2] \right) \end{aligned}$$

* L'operatore $E[\cdot]$ è lineare. Quindi $E[cn_i^2] = cE[n_i^2]$.

BUCKETSORT

Complessità: analisi caso medio

- Si assume che i valori d'input siano distribuiti uniformemente.
- Quindi n valori ripartiti uniformemente in n bucket, dovrebbero dare circa 1 valore per bucket.
- Sia X_{ij} la variabile casuale indicatrice $I\{A[j] \text{ entra nel bucket } i\}$ per $i = 0, \dots, n-1$ e $j = 1, \dots, n$.
- $\Pr\{X_{ij} = 1\} = E[X_{ij}] = 1/n$
- In ciascun bucket i , il numero di elementi è quindi $n_i = \sum_{j=1}^n X_{ij}$

BUCKETSORT

Complessità: analisi caso medio

- La media del costo di INSERTIONSORT sul bucket i è quindi

$$\begin{aligned} E[n_i^2] &= E \left[\left(\sum_{j=1}^n X_{ij} \right)^2 \right] \\ &= E \left[\sum_{j=1}^n X_{ij} \sum_{k=1}^n X_{ik} \right] \\ &= E \left[\sum_{j=1}^n X_{ij}^2 + \sum_{j=1}^n \sum_{k=1 \wedge k \neq j}^n X_{ij} X_{ik} \right] \\ &= \sum_{j=1}^n E[X_{ij}^2] + \sum_{j=1}^n \sum_{k=1 \wedge k \neq j}^n E[X_{ij} X_{ik}] \end{aligned}$$

BUCKETSORT

Complessità: analisi caso medio

- Primo termine: $E[X_{ij}^2] = 1^2 \cdot 1/n + 0^2 \cdot (1 - 1/n) = 1/n$
- Quando $k \neq j$, X_{ij} e X_{ik} sono indipendenti: inserire nel bucket l'elemento j -esimo non condiziona il fatto che anche il k -esimo possa finire nello stesso bucket i .
- Secondo termine: poiché X_{ij} e X_{ik} sono indipendenti, la media di un prodotto è pari al prodotto delle media:

$$\begin{aligned} E[X_{ij}X_{ik}] &= E[X_{ij}]E[X_{ik}] \\ &= \frac{1}{n} \frac{1}{n} \\ &= \frac{1}{n^2} \end{aligned}$$

BUCKETSORT

Complessità: analisi caso medio

- Sostituendo

$$\begin{aligned}E[n_i^2] &= \sum_{j=1}^n E[X_{ij}^2] + \sum_{j=1}^n \sum_{k=1 \wedge k \neq j}^n E[X_{ij} X_{ik}] \\&= n \frac{1}{n} + n(n-1) \frac{1}{n^2} \\&= 1 + \frac{n-1}{n} \\&= 2 - \frac{1}{n}\end{aligned}$$

- Quindi il contributo medio di ciascun bucket al costo di INSERTIONSORT è $O(2 - \frac{1}{n}) = O(1)$

BUCKETSORT

Complessità: analisi caso medio

- Riprendendo l'equazione di partenza:

$$\begin{aligned} E[T(n)] &= E \left[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2) \right] \\ &= \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2]) \\ &= \Theta(n) + nO(1) \\ &= \Theta(n) + O(n) \\ &= \Theta(n) \end{aligned}$$

- Se l'input non ha una distribuzione uniforme, ma la sommatoria dei quadrati delle dimensioni dei bucket è comunque lineare, allora comunque BUCKETSORT ha complessità $\Theta(n)$.

In questa lezione si è visto:

- Il limite inferiore alla complessità computazionale per tutti gli algoritmi di ordinamento basati sul confronto.
- L'esistenza di due algoritmi, COUNTINGSORT e BUCKETSORT, che permettono di ordinare valori numerici in tempo lineare nel caso appropriato: COUNTINGSORT in tempo $\Theta(n + k)$ e BUCKETSORT in tempo medio $\Theta(n)$ sotto opportune ipotesi sulla distribuzione dell'input.

- Studiare cap. 8 escludendo 8.3 (RADIXSORT) [CLRS].
- Risolvere gli esercizi 8.1–1, 8.2–3, 8.2–6, 8.4–2 [CLRS].
- Scrivere una variante dell'algoritmo BUCKETSORT che accetta in input un array di valori decimali positivi senza restrizioni. L'algoritmo non deve modificare i valori in input e deve operare sempre con complessità media $\Theta(n)$ (assumendo l'input distribuito uniformemente).